



Guide for AWS large migrations

AWS Prescriptive Guidance



AWS Prescriptive Guidance: Guide for AWS large migrations

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Introduction	1
Guidance for large migrations	1
About the tools	2
Phases of a large migration	3
Assess phase	4
Mobilize phase	4
Migrate and modernize phase	4
About workstreams	5
About the initialize and implement stages	5
Stage 1: Initializing	7
Step 1: Get ready	8
Step 2: Define rules	9
Step 3: Create portfolio runbooks	9
Step 4: Create migration runbooks	10
Step 5: Evaluate current state	10
Stage 2: Implementing	11
About the migration strategies	14
Retire	14
Retain	15
Rehost	16
Relocate	16
Repurchase	17
Replatform	18
Refactor or re-architect	19
Resources	20
AWS large migrations	20
Additional references	20
Contributors	21
Document history	22

Guide for AWS large migrations

Amazon Web Services ([contributors](#))

February 2022 ([document history](#))

Note

Before reading this guide, we recommend reading [Strategy and best practices for AWS large migrations](#). The strategy discusses best practices for large migrations and provides use cases from customers across various industries. This guide describes a high-level, phased approach for implementing those best practices.

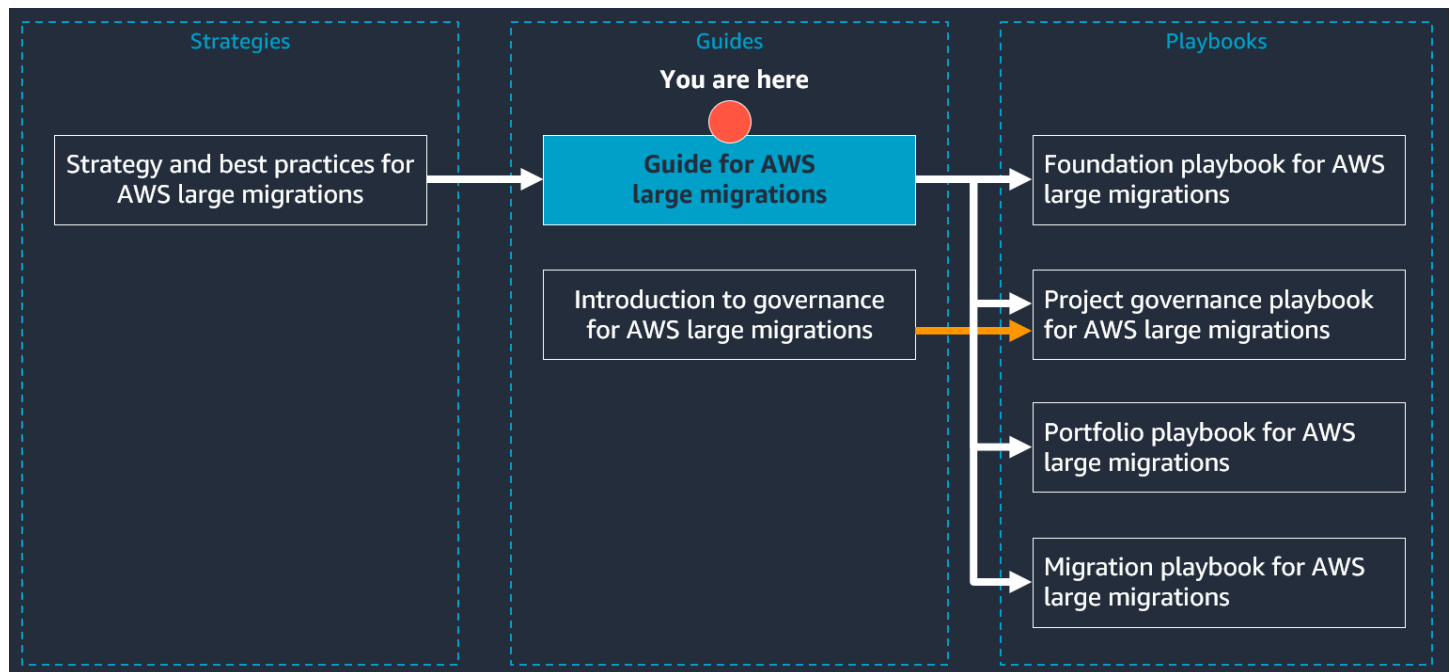
This guide provides an overview of the phased approach for large migrations, which consists of three phases: assess, mobilize, and migrate and modernize. The primary purpose of this guide is to describe the migrate part of the third phase, migrate and modernize, and its two stages, initialize and implement:

- In stage 1, *initialize*, you prepare your platform and people for a large migration. You also define the standard operating procedures (or *runbooks*) for the project. The runbooks and automations simplify and accelerate implementation of the large migration activities in stage 2.
- In stage 2, *implement*, you migrate servers at scale and closely manage and monitor the progress to implement continuous improvements.

Guidance for large migrations

Migrating 300 or more servers is considered a large migration. The people, process, and technology challenges of a large migration project are typically new to most enterprises. This document is part of an AWS Prescriptive Guidance series about large migrations to the AWS Cloud. This series is designed to help you apply the correct strategy and best practices from the outset, to streamline your journey to the cloud.

The following figure shows the other documents in this series. Review the strategy first, then the guides, and then proceed to the playbooks. To access the complete series, see [Large migrations to the AWS Cloud](#).



About the tools

This guide includes a [health-check matrix](#). You can use this tool to assess the health of your migration project throughout the migration. This tool helps you apply best practices and evaluate the efficiency and progress of your large migration project throughout its life cycle.

Additionally, each of the playbooks in this documentation series include templates and tools that can help you build each workstream.

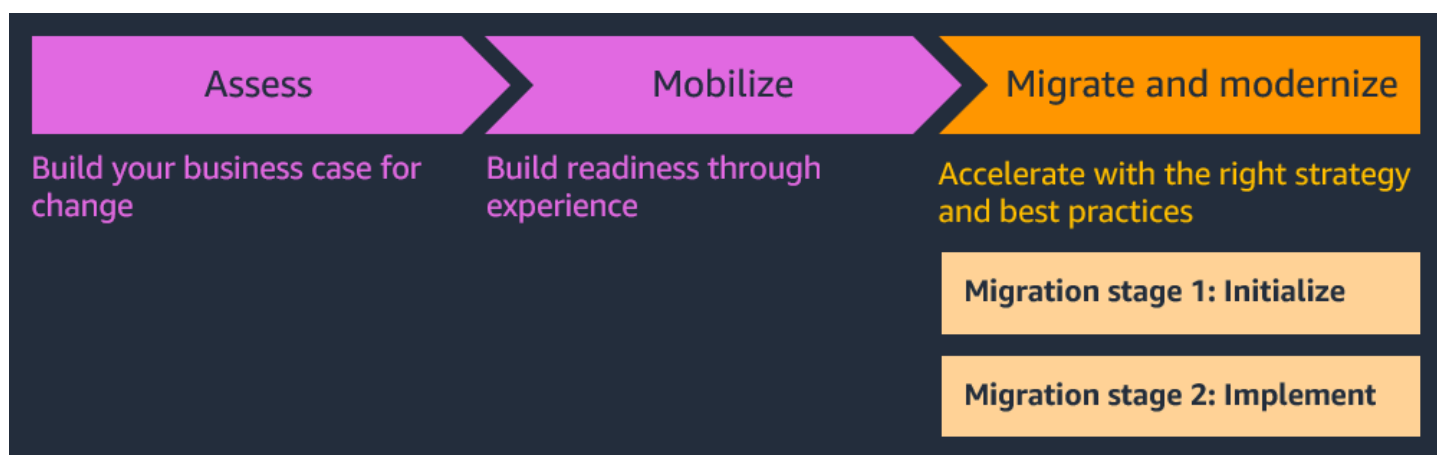
Phases of a large migration

AWS segments the large migration process into three sequential phases:

1. [Assess phase](#) – You build the business case for the migration.
2. [Mobilize phase](#) – You prepare the organization and mobilize the resources needed for the migration.
3. [Migrate and modernize phase of a large migration project](#) – You use your strategy, plan, and the best practices to migrate and modernize. In this phase, migration is divided into two stages, initialize and implement.

The assess and mobilize phases are the foundation of any large migration and prepare the organization for the migration. This guide includes a brief summary of the first two phases and provides references for additional support. The primary purpose of this guide is to describe the migrate part of the third phase, migrate and modernize, and its two stages, initialize and implement. We recommend that you first migrate to the AWS Cloud and then modernize the workload. Modernization is not included in this documentation set for large migrations. For more information about modernization, see [Phased approach to modernizing applications in the AWS Cloud](#) on the AWS Prescriptive Guidance website.

The following figure shows the three phases of a large migration: assess, mobilize, and migrate and modernize. Migration is divided into two stages, initialize and implement.



Even though this document set for large migration is focused on the migrate phase, we highly recommend you go through all three phases of a large migration in sequence. Completing the assess and mobilize phases builds a solid foundation to support the migration.

Assess phase

This is the start of your large-migration journey. In this phase, you build your business case for the large migration. You assess your organization's readiness for cloud transformation, take an initial look at your portfolio, and confirm that the key stakeholders in your organization are aligned.

For more information about the assess phase, see [Evaluating migration readiness](#) on the AWS Prescriptive Guidance website.

Mobilize phase

Now that you have built your business case for a cloud transformation, you use the readiness assessment from the previous phase and begin to fill in the gaps. You mobilize your organization and set up workstreams that prepare your organization for the large migration in the next phase. In this phase, you typically build your AWS landing zone, conduct a more thorough portfolio assessment, build your security and operating model, and prepare teams for change.

For more information about the mobilize phase, see [Mobilize your organization to accelerate large-scale migrations](#) on the AWS Prescriptive Guidance website.

Migrate and modernize phase of a large migration project

After the mobilize phase, your organization should have a solid foundation and be ready to start migration and modernization. Before proceeding, you evaluate the foundations, including the people and the platform, and make sure they are ready to support a large migration. The key for this step is to build standard operating procedures and automations that simplify and accelerate a repeatable pattern. This approach is known as a *migration factory*.

Just like a manufacturing factory, when you build a migration factory, you must initialize and calibrate it, define the standard operating procedures, measure the factory's performance, and continually improve the process and tools. In a large migration, initialization is a critical first stage. Running and improving the factory is the second stage, implementation, which migrates your servers at scale in batches known as waves. You organize your resources into workstreams, which are focused on accomplishing one aspect of the large migration.

This section consists of the following topics:

- [About workstreams](#)

- [About the initialize and implement stages](#)

About workstreams

Each workstream in your large migration project is dedicated to completing certain tasks, and those tasks change as you progress from stage 1, initialization, into stage 2, implementation. Although each workstream is independent, they work together to accomplish the same goal—migrate servers at scale.

The following are the four core workstreams, and you can create additional, supporting workstreams as needed to support your use case:

- **Foundation workstream** – This workstream is focused on preparing the people and platform for the large migration.
- **Project governance workstream** – This workstream manages the overall migration project, facilitates communication, and focuses on completing the project within budget and on time.
- **Portfolio workstream** – The teams in this workstream collect metadata to support the migration, prioritize applications, and perform wave planning.
- **Migration workstream** – Using the wave plan and collected metadata from the portfolio workstream, the teams in this workstream migrate and cutover the applications and servers.

For more information and instructions about how to establish and resource the workstreams in your large migration project, see the [Foundation playbook for AWS large migrations](#).

About the initialize and implement stages

Migrate and modernize is the third and final phase of your migration journey. Generally, you migrate first and then modernize. Migrate consists of two stages:

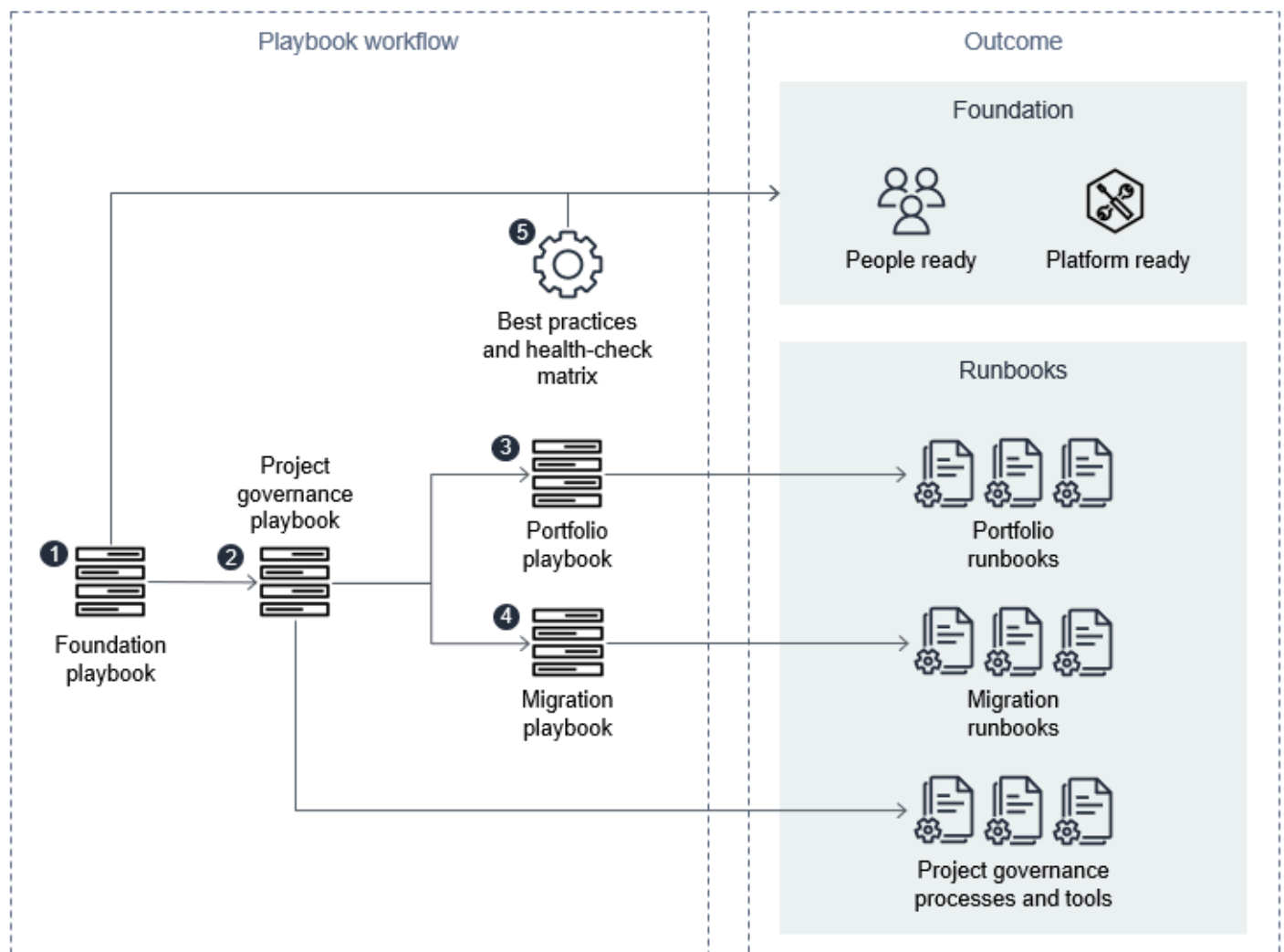
- **Stage 1: Initialize a large migration** – You prepare your platform and people for a large migration. For example, you review the landing zone design and verify that it has enough bandwidth to support the scale, and you design and implement a training plan. In this stage, based on your organization's policies and processes, you also define the standard operating procedures (or *runbooks*) for the large migration. The runbooks and automations simplify and accelerate implementation of the large migration in stage 2.
- **Stage 2: Implement a large migration** – In this stage, you migrate servers at scale by using the runbooks defined in stage 1. You manage the migration factory with project governance tools,

monitor the migration with a health-check matrix, and continuously improve the runbooks in order to increase the velocity of the migration.

Stage	Duration	Purpose
Stage 1: Initialize	1–3 months	• Prepare your platform and people for a large migration. • Build your standard operating procedures (runbooks).
Stage 2: Implement	Varies by project scope and strategy	• Use runbooks to implement the large migration. • Manage, monitor, and improve the migration.

Stage 1: Initializing a large migration

In the initialize stage, it is critical to define the standard operating procedures, or *runbooks*, for most of the migration tasks. Because there are many people, processes, and tools involved in a large migration, you can improve efficiency by defining, connecting, and automating procedures in a runbook. Runbooks provide clear guidance that everyone must follow. Implementing a large migration without runbooks increases the complexity of the migration and might cause the migration effort to lose momentum or fail. To help you get started, this guide reviews the workflow and playbooks, which contain useful templates for building your own runbooks.



The playbooks help you complete the first stage, *initialize*, and prepare for the second stage, *implement*:

1. [Foundation playbook for AWS large migrations](#) – This playbook helps you prepare your infrastructure and people.
2. [Project governance playbook for AWS large migrations](#) – This playbook helps you build project governance processes and tools that effectively manage a large migration project. It includes templates to help you get started.
3. [Portfolio playbook for AWS large migrations](#) – This playbook helps you build your own portfolio runbooks for wave planning and portfolio assessment. It includes templates to help you get started.
4. [Migration playbook for AWS large migrations](#) – This playbook helps you build a runbook for each migration pattern. It includes several templates for different patterns.
5. [Health-check matrix for AWS large migrations](#) – This helps you apply best practices and evaluate the efficiency and progress of your large migration throughout the life cycle of the project.

Step 1: Use the foundation playbook to get ready

In this playbook, you get your platform and people ready.

The following are examples of how you might prepare the people in your organization:

- Make sure your stakeholders are committed to the migration and ready to provide support.
- Define the workstreams in your large migration.
- Define your Cloud Enablement Engine, also known as *Cloud Center of Excellence*, team.
- Create a responsibility assignment model, in the form of a responsible, accountable, consulted, informed (RACI) matrix.
- Establish a training plan for your migration resources.

The following are examples of how you might prepare your organization's platform:

- Get the AWS landing zone ready to support a large migration.
- Evaluate the readiness of your on-premises infrastructure.

For more information, see the [Foundation playbook for AWS large migrations](#).

Step 2: Use the project governance playbook to define the rules

In this step, you define the rules, boundaries, and plans for running the large migration. This includes defining the following plans and processes:

- Communication plan
- Benefit-tracking office
- Escalation process
- Migration and cutover communication gates
- Project management processes and tools

For more information, see [Introduction to governance for AWS large migrations](#) and the [Project governance playbook for AWS large migrations](#).

Step 3: Use the portfolio playbook to create portfolio runbooks

Now, you get a better understanding of the scope and strategy. In this step, you follow the portfolio playbook to build detailed runbooks for portfolio assessment and wave planning. The goal is to make sure the migration factory doesn't run out of raw materials (servers) and has all the information needed to support the migration. Tasks include the following:

- Review the migration strategy from the mobilize phase (if you defined it in that phase). For more information, see [About the migration strategies](#).
- Define metadata requirements for each migration pattern.
- Define the metadata collection process and a metadata storage location that serves as a single source of truth.
- Define application prioritization rules.
- Define application deep dive rules.
- Define the wave planning process.

For more information, see the [Portfolio playbook for AWS large migrations](#).

Step 4: Use the migration playbook to build a migration runbook for each pattern

This step can run in parallel with the previous step. Using the migration playbook, you build your runbook for each of the migration patterns, such as rehosting to Amazon Elastic Compute Cloud (Amazon EC2) with AWS Cloud Migration Factory, or replatforming storage to Amazon Elastic File System (Amazon EFS) with AWS DataSync. You repeatedly test the runbooks until there are minimal to no errors. Tasks include the following:

- Understand the migration patterns.
- Get the runbook templates and add your on-premises process and tools.
- Test and improve the runbooks.
- Automate the runbooks as much as possible.

For more information, see the [Migration playbook for AWS large migrations](#).

Step 5: Use the health-check matrix to evaluate your current state

In this final step, you verify that your project team has completed the previous steps and identify any remaining gaps. Evaluating the health of the migration and improving the process are critical to the migration's success and helps maintain alignment with best practices.

Use the [health-check matrix](#) to evaluate the current state of your migration from a people, process, and technology perspective. This is not a one-off process. You should perform this evaluation on a regular basis.

Stage 2: Implementing a large migration

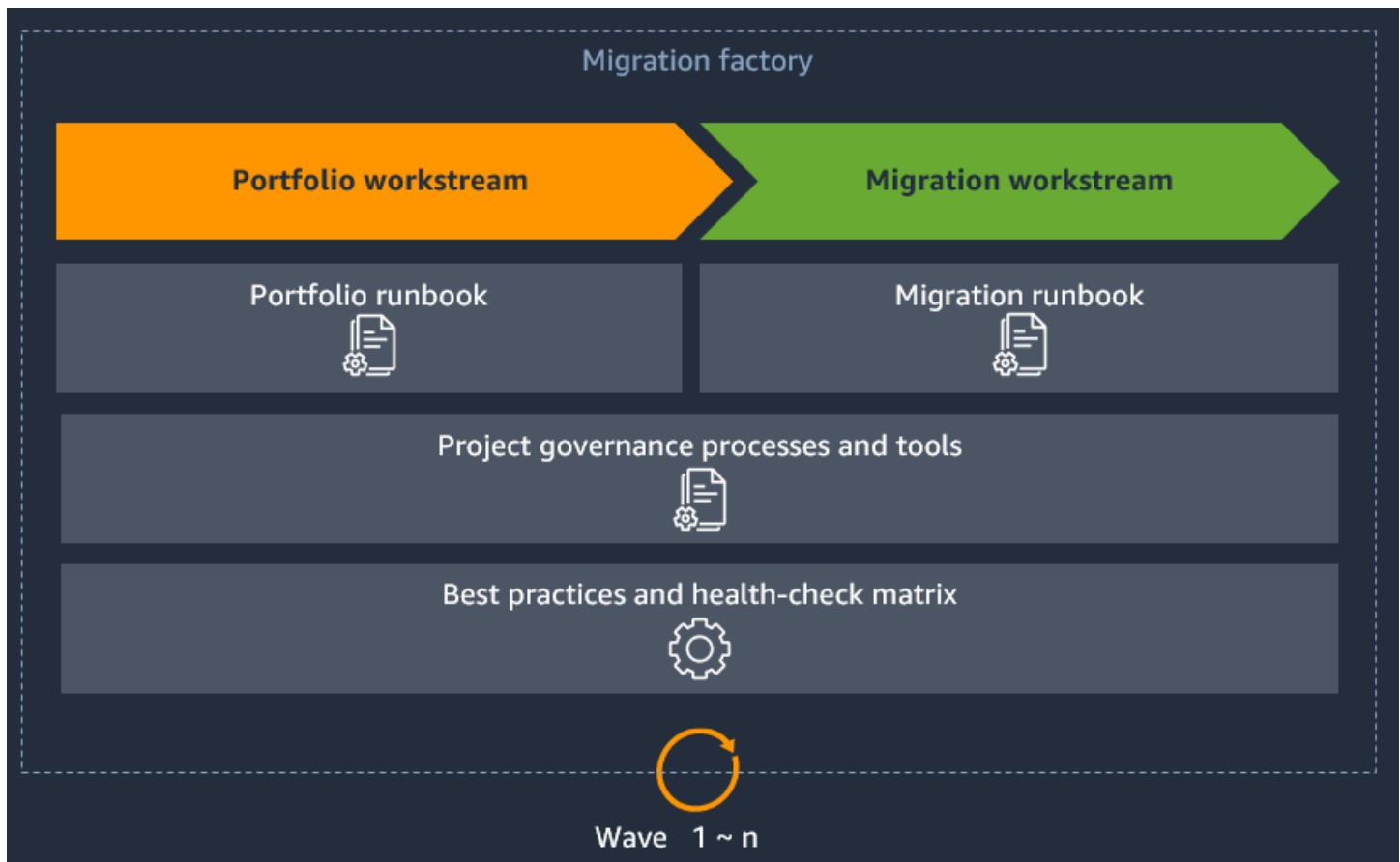
In stage 2 of a large migration, the goal is to migrate your servers at scale. For example, to migrate 1,000 servers in 6 months, you might start by migrating 5 servers per week and then gradually increase the velocity up to 50–100 servers per week.

Now you use the runbooks that you developed in stage 1 to migrate servers in waves. The first couple of waves are typically small because the migration and portfolio workstreams are adopting and adjusting the processes in their runbooks. Improving the runbooks is key to the success of a large migration. Runbooks are living documents. You must review, revise, and improve your runbooks after each cutover. As the runbooks evolve over time, the velocity should increase with every wave.

In stage 2, you use the following components to operate the migration factory:

- **Project governance rules** – You follow the project governance processes to manage waves, communication, timelines, and cutovers. These processes and tools make sure that everyone does the right thing at the right time and in the right order.
- **Portfolio runbooks** – You use the portfolio runbooks to prioritize applications, plan waves, and collect the required metadata that supports the migration. This metadata is the equivalent of raw materials in a manufacturing factory.
- **Migration runbooks** – You use the migration runbooks to migrate apps and servers, load the metadata to your migration tools, and complete the cutover process at the end of each wave. When following the migration runbooks, you adhere to the wave plan in the portfolio runbooks and use the metadata in the portfolio runbooks or from another single source of truth.
- **Large migration best practices and health-check matrix** – You use the health-check matrix to evaluate your current state frequently and regularly to make sure that everything is on track.

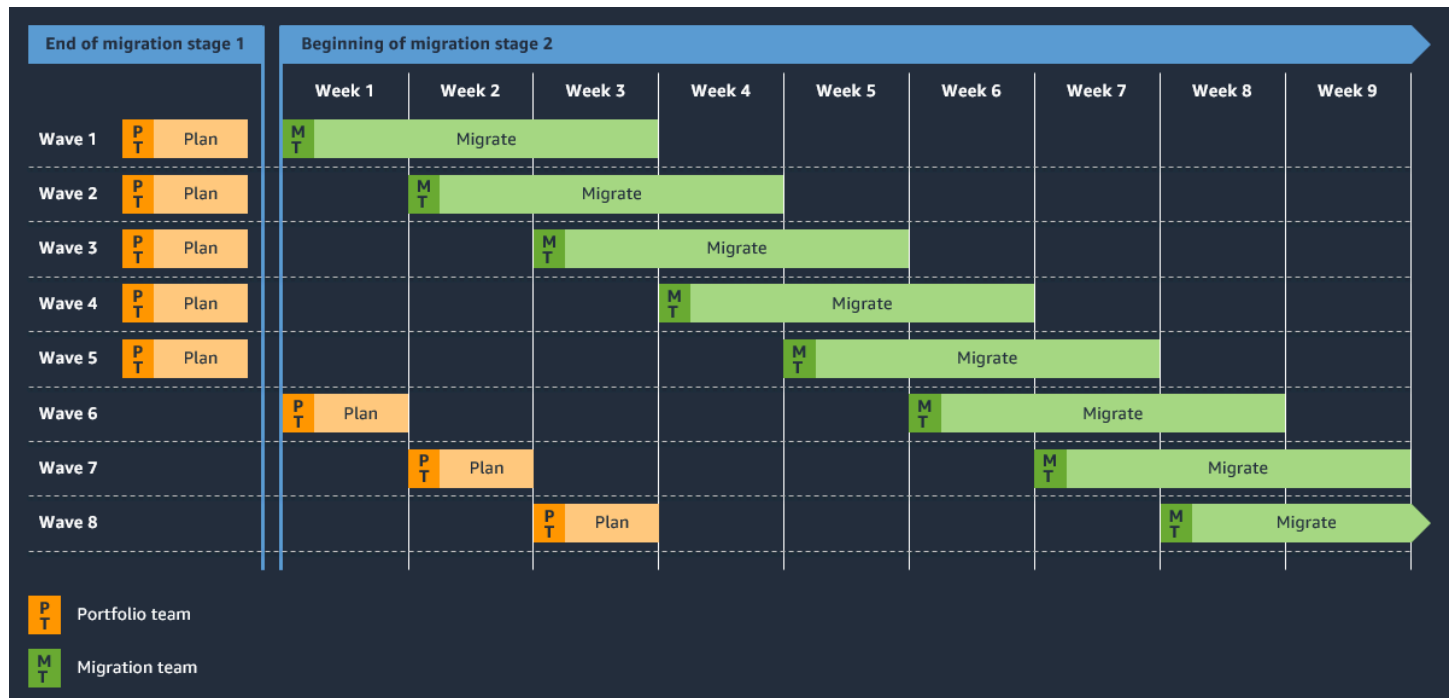
The following figure shows a typical migration factory for large migrations.



The runbooks are the key components of the migration factory, and they work together to form a data flow through two workstreams, portfolio and migration. For more information about these workstreams, see the [Foundation playbook for AWS large migrations](#). Rather than seeing a wave all the way through the migration factory, teams are usually dedicated to certain parts of the factory, and the waves flow through each workstream. The duration of each workstream varies based on your project timeline, scope, and resource availability. For example, the portfolio workstream might be 3 weeks, and the migration workstream might be 2–5 weeks. Prevent supply chain problems in your migration factory by ensuring that there are sufficient server waves lined up for migration. We recommend that the portfolio workstream is five waves ahead of the migration workstream.

The following figure shows a dynamic view of a typical migration factory. For each wave, the portfolio workstream runs 1–2 weeks, and the migration workstream typically runs 3–4 weeks. The portfolio workstream is five waves ahead of the migration workstream, so there is always a five-wave buffer between the portfolio and migration workstreams. At the end of migration stage 1, initializing, the portfolio workstream completes wave planning for a buffer of five waves. When the migration workstream starts migrating applications, this indicates that you have entered stage 2,

implementing. Both the portfolio and the migration workstreams continue to process waves, and the buffer prevents the migration workstream from running out of servers to migrate.



About the migration strategies

A *migration strategy* is the approach used to migrate a workload into the AWS Cloud. There are seven migration strategies for moving applications to the cloud, known as the 7 Rs:

- [Retire](#)
- [Retain](#)
- [Rehost](#)
- [Relocate](#)
- [Repurchase](#)
- [Replatform](#)
- [Refactor or re-architect](#)

Common strategies for large migrations include rehost, replatform, relocate, and retire. Refactor is not recommended for large migrations because it involves modernizing the application during the migration. This is the most complex of the migration strategies, and it can be complicated to manage for a large number of applications. Instead, we recommend rehosting, relocating, or replatforming the application and then modernizing the application after the migration is complete.

Selecting migration strategies is critical to a large migration. You might have selected migration strategies in the mobilize phase or during the initial portfolio assessment. This section reviews each migration strategy and their common use cases.

Retire

This is the migration strategy for the applications that you want to decommission or archive. Retiring the application means that you can shut down the servers within that application stack. The following are common use cases for the retire strategy:

- There is no business value in retaining the application or moving it to cloud.
- You want to eliminate the cost of maintaining and hosting the application.
- You want to reduce the security risks of operating an application that uses an operating system (OS) version or components that are no longer supported.

- You might want to retire applications based on their performance. For example, you might want to retire applications that have an average CPU and memory usage below 5 percent, known as *zombie applications*. You might also choose to retire some applications that have an average CPU and memory usage between 5 and 20 percent over a period of 90 days, known as *idle applications*. You can use the utilization and performance data from your discovery tool to identify zombie and idle applications.
- There has been no inbound connection to the application for the last 90 days.

For more information, see [Best practices for assessing applications to be retired during a migration to the AWS Cloud](#).

Retain

This is the migration strategy for applications that you want to keep in your source environment or applications that you are not ready to migrate. You might choose to migrate these applications in the future.

The following are common use cases for the retain strategy:

- **Security and compliance** – You might want to retain applications in order to remain in compliance with data residency requirements.
- **High risk** – You might decide to retain an application because it requires a detailed assessment and plan prior to migration.
- **Dependencies** – You might decide to retain an application if you need to migrate one or more other applications first.
- **Applications that are recently upgraded** – You might want to postpone migrating the application until the next technical refresh because you recently invested in upgrading your current system.
- **No business value to migrate** – There is no business value for migrating some applications to the cloud, such as those with only a few internal users.
- **Plans to migrate to software as a service (SaaS)** – You might choose retain an application until the SaaS version is released by the vendor. This is a common strategy for vendor-based applications.
- **Unresolved physical dependencies** – You might choose to retain an application that is dependent on specialized hardware that does not have a cloud equivalent, such as machines in a manufacturing plant.

- **Mainframe or mid-range applications and non-x86 Unix applications** – These applications require careful assessment and planning before migrating them to the cloud. Examples of mid-range applications include IBM AS/400 and Oracle Solaris.
- **Performance** – You might want to retain applications based on their performance. For example, you might want to keep zombie or idle applications in your source environment.

Rehost

This strategy is also known as *lift and shift*. Using this strategy, you move your applications from your source environment to the AWS Cloud without making any changes to the application. For example, you migrate your application stack from on-premises to the AWS Cloud.

With rehost, you can migrate a large number of machines from multiple source platforms (physical, virtual, or another cloud) to the AWS Cloud without worrying about compatibility, performance disruption, long cutover windows, or long-distance data replications.

Your application continues to serve users while the workloads are being migrated, which minimizes disruption and downtime. The downtime depends on your cutover strategy.

This strategy helps you to scale your applications without implementing any cloud optimizations that could save you time or money. Applications are easier to optimize or re-architect when they are already running in cloud because it is easier to integrate to AWS services and manage your workloads.

You can automate rehosting by using the following services:

- [AWS Transform MGN](#)
- [AWS Cloud Migration Factory Solution](#)
- [VM Import/Export](#)

For a list of migration patterns for the rehost migration strategy, see [Rehost](#) on the AWS Prescriptive Guidance website.

Relocate

Using this strategy, you can transfer a large number of servers, comprising one or more applications, at a given time from on-premises platform to a cloud version of the platform. You

can also use the relocate strategy to move instances or objects to a different virtual private cloud (VPC), AWS Region, or AWS account. For example, you can use this strategy to transfer an Amazon Relational Database Service (Amazon RDS) DB instance to another VPC or AWS account.

The relocate strategy doesn't require that you purchase new hardware, rewrite applications, or modify your existing operation. During relocation, the application continues to serve users, which minimizes disruption and downtime. Relocate is the quickest way to migrate and operate your workload in the cloud because it does not impact the overall architecture of your application.

For a list of migration patterns for the relocate migration strategy, see [Relocate](#) on the AWS Prescriptive Guidance website.

Repurchase

This strategy is also known as *drop and shop*. You replace your application with a different version or product. The new application should provide more business value than the existing, on-premises application, including features such as accessibility from anywhere, no infrastructure to maintain, and pay-as-you-go pricing models. Repurchasing the application typically reduces costs associated with maintenance, infrastructure, and licensing.

The following are common use cases for the repurchase migration strategy:

- **Moving from a traditional license to SaaS** – This removes the burden of managing and maintaining the infrastructure and helps reduce licensing issues.
- **Version upgrades or third-party equivalents** – By replacing your existing on-premises application with the vendor's latest version or third-party equivalent in the cloud, you can leverage new features, integrate with cloud services, and scale the application more easily.
- **Replacing a custom application** – You can avoid recoding and re-architecting a custom application by repurchasing a vendor-based SaaS or cloud-based application.

Before purchasing, you need to assess the application according to your business requirements, especially security and compliance.

After you purchase the new application, the following are the next steps:

- Training your team and users with the new system
- Migrating your data to the newly purchased application

- Integrating the application into your authentication services, such as Microsoft Active Directory, to centralize authentication
- Configuring networking to help secure communication between the purchased application, your users, and your infrastructure

Typically, the application vendor helps you with these activities for a smooth transition.

Replatform

This strategy is also known as *lift, tinker, and shift* or *lift and reshape*. Using this migration strategy, you move the application to the cloud, and you introduce some level of optimization in order to operate the application efficiently, to reduce costs, or to take advantage of cloud capabilities. For example, you might replatform a Microsoft SQL Server database to Amazon RDS for SQL Server.

Using this strategy, you might make a few or many changes to the application, depending on your business goals and your target platform.

The following are common use cases for the replatform migration strategy:

- You want to save time and reduce cost by moving to a fully managed service or serverless service in the AWS Cloud.
- You want to improve your security and compliance stance by upgrading your operating system to the latest version.
- You can reduce costs by using [AWS Graviton Processors](#), custom-built processors developed by AWS.
- You can reduce costs by moving from a Microsoft Windows operating system to a Linux operating system. You can port your .NET Framework applications to .NET Core, which can run on a Linux operating system. [Porting Assistant for .NET](#) is an analysis tool that helps you port your applications to Linux.
- You can improve performance by migrating virtual machines into containers, without making any code changes. You can modernize your .NET and Java applications into containerized applications by using the [AWS App2Container migration tool](#).

The replatform strategy keeps your legacy application running without compromising security and compliance.

Replatform reduces cost and improves performance by migrating to a managed or serverless service, moving virtual machines to container, and avoiding licensing expenses.

For a list of migration patterns for the replatform migration strategy, see [Replatform](#) on the AWS Prescriptive Guidance website.

Refactor or re-architect

Using this strategy, you move an application to the cloud and modify its architecture by taking full advantage of cloud-native features to improve agility, performance, and scalability. This is driven by strong business demand to scale, accelerate product and feature releases, and to reduce costs.

The following are common use cases for the refactor migration strategy:

- The legacy mainframe application can no longer address the demand of the business due to its limitations or is expensive to maintain.
- You have a monolith application that is already hindering efforts to deliver product quickly or address customer needs and demands.
- You have a legacy application that nobody knows how to maintain, or the source code is unavailable.
- The application is difficult to test, or test coverage is very low. This affects the quality and delivery of new application features and fixes. By redesigning the application for the cloud, you can increase the test coverage and integrate automated testing tools.
- For security and compliance reasons, when moving a database to the cloud, you might need to extract some tables (such as customer information, patient, or patient diagnosis tables) and retain those tables on premises. In this situation, you need to refactor your database in order to separate the tables that will be migrated from those that will be retained on premises.

For a list of migration patterns for the refactor migration strategy, see [Re-architect](#) on the AWS Prescriptive Guidance website.

Resources

AWS large migrations

To access the complete AWS Prescriptive Guidance series for large migrations, see [Large migrations to the AWS Cloud](#).

Additional references

- [AWS Cloud Migration Factory Solution](#)
- [When to Choose AWS Transform MGN](#)
- [Best practices for assessing applications to be retired during a migration to the AWS Cloud](#)
- [AWS Prescriptive Guidance migration patterns](#)

Contributors

The following individuals contributed to this document:

- Wally Lu, Principal Consultant
- Damien Renner, Senior Migration Specialist

Document history

The following table describes significant changes to this guide. If you want to be notified about future updates, you can subscribe to an [RSS feed](#).

Change	Description	Date
Updated name of AWS solution	We updated the name of the referenced AWS solution from <i>CloudEndure Migration Factory</i> to <i>Cloud Migration Factory</i> .	May 2, 2022
Initial publication	—	February 28, 2022